
Assignment 2
Classification and
Prediction
Machine Learning I

**Dual Bachelor in Data Science and Engineering
and Telecommunications Engineering**

Alejandro Barroso - 100499081

Juan José Rosales - 100499176

Group 196

Index

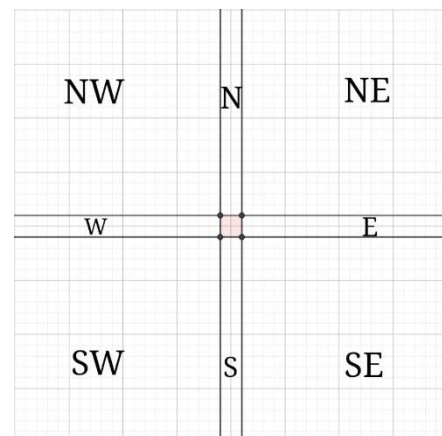
Phase 1: Selection of the state information and reward function.....	3
Phase 2: Generation of the agent.....	4
Phase 3: Enhancement of the agent.....	5
Conclusions.....	6

Phase 1: Selection of the state information and reward function

In this first phase we focus our attention on selecting the states and the reward function of our model.

To select an optimal number of states we look at the most important variables determining the next action that the snake should take. The possible actions of the snake are UP, DOWN, RIGHT or LEFT. Besides, to avoid a variable number of them, we must NOT use attributes related to variable quantities like the size of the screen. Having this in mind the most relevant attributes for the desired goal are the actual direction of the snake (direction), the relative position between snake and apple.

Our first attempt used the relative position of the food from the snake to generate all the possible states. We created the attribute **rel_pos** with 8 possible values: N, S, E, W, NE, NW, SE and SW. However, during the phase 2 we saw that using only these states were not enough to represent the game correctly and some changes were needed.



Therefore, we decided to add the attribute **direction** can take the values UP (1), DOWN (2), RIGHT (3), and LEFT (4). Moreover, the **rel_pos** was divided into 2 simpler attributes: **rel_x**, that can take the values 0 (food is in same x coordinate), 1 (food is right), 2 (food is left); and **rel_y**, that works as **rel_x** but for the y axis.

Moving forward, now we must take care of the reward function. The objective is to reward the model when performing good and penalize it when not performing in the expected way. We also want to emphasize different levels of actions. The model can do a very good movement like eating an apple, which is the objective, or a good but simpler action like moving towards the apple. The same but in negative terms when dying or moving away from the apple. By giving a reward for reducing the distance to the food and subtracting it when the snake gets further from it, we are forcing the snake to take the fastest path to the food. Therefore, we decided that the reward function should be of this form:

- Eating an apple: +10
- Moving towards apple: +1
- Moving away from the apple: -1
- Dying: -10

Phase 2: Generation of the agent

The first thing we had to do was to complete the methods that were empty in the template code of AulaGlobal. We started with the **calculate_reward** function, which was easy to implement since we already had the idea of the different rewards that would be given for each action. However, to know if the snake was moving towards or away from the apple, we created a method called **compute_dist** to compute the distance at each tick of the game. The current distance value was saved in another variable called **previous_distance** used in the next tick to be able to see if the snake moved forward or away from the food.

Besides, we completed the **update_q_table** function. To do so we used the formulas for the ϵ -greedy:

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a')]$$

This was very easy due to the work done in tutorial 4, where the same function was used but with very slight changes (the name of `self.gamma` was `self.discount`).

Finally, we implemented the **get_state** function. Since in the previous phase we decided to use the attributes `direction`, `rel_x` and `rel_y` to define each state of the game, we needed to calculate them with some simple algorithms. Then, we needed to get the states from them, so we had to join these values in a way that we have a state for each possible combination of attributes. Thus, the number of each state will be computed as follows:

```
State = direction (4 possible values) + rel_x (3 possible values) * 4 +  
        rel_y (3 possible values) * 12
```

Doing it in this way will avoid having the same state for different combination of features. In total, the number of states increased to 36. With this information we were able to compute the Q-table size, which is the number of states by the number of actions. In our case the possible states are 36 and the number of actions is 4. Therefore, our Q-table is a [36x4] matrix.

We also had to define the best values for α , ϵ and γ attributes. To do so, we did several different combinations of them and trained the agent from scratch for each tune of parameters. We concluded that the best outcome was obtained when $\alpha = 0.1$, $\epsilon = 0.01$ and $\gamma = 0.9$

Finally, we did an evaluation of our agent. After training it with many episodes in different screen sizes, we turned the training variable to False and saw how the snake performed. Since the snake did not grow when eating food, and was always following the food's position, it never died. Therefore, we achieved what we expected in this phase.

Phase 3: Enhancement of the agent

The goal of this phase is to improve our model. There are two ways in which we can improve the final score of the model, eating apples in a faster way and preventing the snake from dying.

To accomplish it, we must prevent the snake from colliding with limits or with its own body. To implement it, we thought about adding a few more attributes that show whether the snake has in its vicinity an obstacle (wall or body) or not. With these new attributes, the states will include information about obstacles in each direction.

To do so, our first approach was elaborating a new attribute called **collision** that has different values depending on the location of the obstacle. We implemented it using several elifs for each of the 4 possible directions in which the obstacle can be. However, after trying to train the snake with this new variable we discovered that it only detected one of the possible collisions due to the elif structure, when the snake could have obstacles in several directions at the same time.

Our next approach was to generate 4 collision features, one for each direction. They would have a value of 0 if there were no obstacles in that direction and 1 otherwise. These new attributes were much more representative of the game, however, we wanted to reduce the number of attributes.

Finally, we decided to simplify the number of collision-attributes by having just 2 variables instead of 4. The new variables were `collision_x` and `collision_y` and could take values from 0 to 3:

Collision_x	Value	Collision_y	Value
No collisions	0	No collisions	0
Collision RIGHT	1	Collision UP	1
Collision LEFT	2	Collision DOWN	2
Collision RIGHT & LEFT	3	Collision UP & DOWN	3

```
State = direction (4 possible values) + rel_x (3 possible values) * 4 +  
rel_y (3 possible values) * 12 + collision_x (4 values) * 36 + collision_y (4 values) * 144
```

With this new set of attributes, the number of possible states increased to 576, so the Q-table now has size [576x4]. Besides, we repeated the tuning of the Q-learning parameters (α , ϵ and γ) in the training, as we did in phase 2. However, we obtained the same results as in the previous phase.

Finally, after training the agent we could evaluate its performance. The results were almost perfect, since the agent was able to take the best path to reach the food, while dodging its own body. The only way in which it dies is when it enters a loop created by itself with no possible way out.

Conclusions

In conclusion, through the described process above, we successfully constructed an agent capable of playing the Snake game focusing on efficiency and strategic movement. To achieve this, we solved in order the following problems: choosing the optimal number of states, the optimal reward function, and the optimal parameters. We chose relevant state attributes capturing the essential aspects of the game, such as the snake's direction and the relative position to the apple. We combined them to obtain the states representing the main logic of the game and possible actions in each of them. Our reward function was designed to incentivize behaviors leading to increasing the score of the game, like approaching the apple, while penalizing detrimental actions, like moving away from the apple or dying. Then, by trial and error we selected the best parameters to optimize the learning process.

After having all this information of the desired model, we implemented crucial functions of the agent, including reward calculation and Q-table updates, following the ϵ -greedy approach for exploration and exploitation. It was helpful that by defining states based on direction, relative x, and y positions, we ensured a comprehensive representation of the game's dynamics, leading to effective decision-making, even from the beginning. Finally, with the goal of preventing collisions with obstacles, we augmented the state representation space with collision attributes, allowing the agent to make informed decisions about its surroundings. Despite the increase in state space dimensionality, our agent maintained its effectiveness, highlighting the robustness of our approach. After all this, we ended up with a good reinforcement learning model able to play the snake game with a high rate of success.